

Java Operators

Java Operators

- 1. Arithmetic Operators
- 2. Relational Operators
- 3. Bitwise Operators
- 4. Shift Operators
- 5. Logical Operators
- 6. Assignment Operator
- 7. The Ternary Operator (Preview)
- 8. Operator Precedence

★ One-Page Revision

📝 Practice Questions

🔥 Quick FAQ

Java Operators

Arithmetic, Relational, Bitwise, Logical, Shift & Assignment Operators

1. Arithmetic Operators

The five basic arithmetic operators work exactly as in mathematics:

Operator	Meaning	Example (a=5, b=10)	Result
+	Addition	a + b	15
-	Subtraction	a - b	-5
*	Multiplication	a * b	50
/	Division	b / a	2
%	Modulus (remainder)	b % a	0

```
int a = 5, b = 10;
int c = a + b;    // 15
int d = a - b;    // -5
int e = a * b;    // 50
int f = b / a;    // 2
int g = b % a;    // 0
```

Compound Assignment Shortcuts

Java lets you combine an arithmetic operator with `=` to shorten "take a variable, do math on it, and reassign it to itself":

Long form	Shortcut
<code>h = h + 2</code>	<code>h += 2</code>
<code>h = h - 2</code>	<code>h -= 2</code>
<code>h = h * 3</code>	<code>h *= 3</code>
<code>h = h / 5</code>	<code>h /= 5</code>
<code>h = h % 5</code>	<code>h %= 5</code>

✅ **Best Practice:** In real-world code, the compound form (`h += 2`) is almost always preferred over the long form — it's shorter and slightly faster to write.

Increment (`++`) and Decrement (`--`)

Since incrementing/decrementing by exactly 1 is so common in programming (loops especially), Java gives it a dedicated operator:

```
i++;    // same as i = i + 1
i--;    // same as i = i - 1
```

🔴 **Important:** There is no `**` (multiply-by-itself) or `///` shortcut operator — increment/decrement only exist for `+1` / `-1` because those are overwhelmingly the most common operations (e.g. moving to the next memory slot in a loop).

Pre-Increment vs. Post-Increment

Both `j++` (postfix) and `++j` (prefix) increment `j` by 1 — but they differ in **when** the increment happens relative to the assignment:

Form	Name	Behavior
<code>k = j++</code>	Postfix	Assign the current value of <code>j</code> to <code>k</code> first, then increment <code>j</code>
<code>l = ++j</code>	Prefix	Increment <code>j</code> first , then assign the new value to <code>l</code>

Worked Example (`j` starts at `9`):

```
int j = 9;
int k = j++;    // k = 9 (old value), then j becomes 10
int l = ++j;    // j becomes 11 first, then l = 11
```

Variable	Value
<code>j</code> after <code>k = j++</code>	<code>10</code>
<code>k</code>	<code>9</code> (the value <i>before</i> incrementing)
<code>j</code> after <code>l = ++j</code>	<code>11</code>
<code>l</code>	<code>11</code> (the value <i>after</i> incrementing)

💡 Easy Hinglish Explanation

"Postfix (`j++`) ka matlab: pehle jo bhi value hai wahi use kar lo, uske baad variable ko badhao. Prefix (`++j`) ka matlab: pehle variable ko badhao, phir badhi hui value ko use karo. Same rule minus-minus (`--`) ke liye bhi lagta hai."

⚠ **Common Mistake:** Assuming `j++` and `++j` always behave identically because "both just increment j." They only differ when the result is being **assigned or used in the same expression** — standing alone on their own line, both do the exact same thing.

2. Relational Operators

Relational operators **compare** two values and always produce a **boolean** result (`true` or `false`).

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code><</code>	Less than
<code>></code>	Greater than
<code><=</code>	Less than or equal to
<code>>=</code>	Greater than or equal to

```
int a = 5, b = 10;
boolean c = (a == b); // false - is a equal to b?
boolean d = (a != b); // true  - is a not equal to b?
boolean e = (a < b);  // true
boolean f = (a > b);  // false
```

⚠ Critical Distinction: `=` vs `==`

- `a = b` → **Assignment operator**. Takes whatever value is in `b` and stores it into `a`. After this, `a`'s old value is gone.
- `a == b` → **Relational operator**. Asks the compiler a *question*: "is `a` equal to `b`?" Returns `true` or `false`, and neither variable's value changes.

Confusing these two is one of the most common beginner bugs in C-family languages.

Any expression using a relational operator — like `a < b` — is called a **boolean expression**, and these are used constantly in `if` statements and loop conditions.

3. Bitwise Operators

Bitwise operators work directly on the **individual bits** (0s and 1s) of a number — not on the whole decimal value.

Operator	Name	Symbol
AND	Bitwise AND	<code>&</code>
OR	Bitwise OR	<code> </code>
XOR	Bitwise Exclusive OR	<code>^</code>
NOT	Bitwise NOT (unary)	<code>~</code>

🔴 **Important:** `&`, `|`, and `^` are **binary operators** (need two operands). `~` is a **unary operator** (needs only one) — same category as `++`.

Truth Table

Using `0 = false` and `1 = true` for intuition:

A	B	A & B (AND)	A B (OR)	A ^ B (XOR)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

- **AND (&)**: Result bit is `1` only if **both** bits are `1`.
- **OR (|)**: Result bit is `1` if **at least one** bit is `1`.
- **XOR (^)**: Result bit is `1` only if the number of `1`s is **odd** (i.e. the two bits are *different*).
- **NOT (~)**: Flips every bit — `0` becomes `1`, `1` becomes `0`.

Worked Example: `a = 2`, `b = 3` (8-bit representation)

```
a = 2 → 00000010
b = 3 → 00000011
```

Operation	Bit-by-bit	Result (binary)	Result (decimal)
<code>a & b</code>	00000010	00000010	2
<code>a b</code>	00000011	00000011	3
<code>a ^ b</code>	00000001	00000001	1
<code>~a</code>	invert all bits of <code>a</code>	11111101	-3

 **Important:** `~a` produces `-3`, not a simple flip you'd expect visually — because the Most Significant Bit becomes `1`, making the compiler interpret the result as a **negative number** (two's complement).

4. Shift Operators

Shift operators move all the bits in a number left or right by a specified number of positions.

Operator	Name	Effect
<code><<</code>	Left Shift	Multiplies the number by 2, per position shifted
<code>>></code>	Right Shift (signed)	Divides the number by 2, preserves the sign
<code>>>></code>	Right Shift with Zero Fill (unsigned)	Divides the number by 2, always fills with <code>0</code> — ignores sign

Left Shift (<<) — Worked Example

```
byte b = 8;
b = (byte) (b << 1);
```

`8` in binary (8-bit) is `00001000`. Shifting left by 1 moves every bit one position left, and a new `0` fills the empty spot on the right:

```
00001000 → 00010000 (= 16)
```

★ **Exam Tip:** `x << 1` is equivalent to `x * 2`. `x << n` is equivalent to `x * 2n`.

What happens if you keep shifting? `8 → 16 → 32 → 64 → 128`. But `128` doesn't fit in a signed `byte` (`byte` range is `-128` to `127`) — the bit lands on the **MSB**, so the compiler reads the result as **negative**: it becomes `-128`. Keep shifting once more, and every bit falls off the edge — the result becomes `0`.

Shift Operators Only Work on `int` and `long`

⚠ **Common Mistake** — this trips up almost everyone:

Shift operations (`<<`, `>>`, `>>>`) **never actually operate on `byte` or `short`**. Just like arithmetic, Java **type-promotes** `byte` / `short` to `int` first, performs the shift on the full 32-bit `int`, and only truncates back down to 8 bits when you cast the result back to `byte`.

This means the "extra" bits that appear to fall off the edge of a `byte` **aren't actually lost** — they're still sitting inside the promoted `int`. If you shift a promoted `byte` value and store the result in an `int` (not casting back to `byte`), you'll see the full, untruncated result — which can look surprisingly large or negative compared to what you'd expect from an 8-bit view.

Worked Example: `byte b = 1;` shifted left 7 times, viewed as a `byte`, shows `0` (since only the last 8 bits are kept, and the `1` bit moved into position 8 which gets truncated away). But if you instead assign the promoted result to an `int` without casting, you'll see the true value: `-256` — because internally, the number is 32 bits wide, and the `1` bit is sitting in the 9th position, making the full 32-bit number negative.

The Shift Amount Wraps Around (mod 32 / mod 64)

For `int` (32-bit), the maximum meaningful shift is **31 positions** — shifting an LSB all the way up to the MSB. Shifting further would push data completely off the edge with no benefit, so Java **wraps the shift amount using modulus**:

```
int shift amount → effectively (shift amount % 32)
long shift amount → effectively (shift amount % 64)
```

★ **Exam Tip** — memorize this:

`i << 32` is equivalent to `i << 0` (i.e. no shift at all), since `32 % 32 = 0`. `i << 33` is equivalent to `i << 1`, since `33 % 32 = 1`. `i << 31` gives the **most negative possible `int`** value (equivalent to `Integer.MIN_VALUE`), since the single `1` bit lands exactly on the MSB.

Right Shift, Signed (`>>`) vs. Unsigned (`>>>`)

Both move bits to the right and discard the LSB. The difference is **what fills the new MSB slot**:

Operator	New MSB bit	Effect on sign
<code>>></code> (signed)	Copies the original MSB (preserves sign)	A negative number stays negative
<code>>>></code> (unsigned)	Always inserts <code>0</code>	A negative number can become positive

Example: For a negative `byte` value like `-128` (`10000000`), a normal right shift (`>>`) keeps re-inserting `1` at the MSB to preserve the negative sign, giving `-64`. But an unsigned right shift (`>>>`) always inserts `0`, turning it into `+64` instead.

💡 **Easy Hinglish Explanation**

" `>>` hamesha original sign yaad rakhta hai — agar number negative tha, toh naya bit bhi 1 hi aayega taaki number negative hi rahe. `>>>` bilkul care nahi karta sign ka — hamesha 0 fill karega, chahe number pehle negative ho ya positive."

🔴 **Important — practical use case:** `>>>` is useful when negative values don't make conceptual sense at all — for example, image pixel values, which should always stay non-negative regardless of bit manipulation.

Compound Assignment for Bitwise/Shift Operators

Same pattern as arithmetic operators — every bitwise/shift operator has a compound form:

```
c &= 2;    // same as c = c & 2
c |= 2;    // same as c = c | 2
c ^= 2;    // same as c = c ^ 2
c >>= 1;   // same as c = c >> 1
c <<= 1;   // same as c = c << 1
c >>>= 1;  // same as c = c >>> 1
```

5. Logical Operators

Operator	Name
<code>&&</code>	Logical AND
<code> </code>	Logical OR
<code>!</code>	Logical NOT (unary)

The **rules** are identical to the bitwise truth table above (both need **true+true** for AND to be true; OR needs at least one true) — but there's a critical structural difference:

	Bitwise (<code>&</code> , <code> </code>)	Logical (<code>&&</code> , <code> </code>)
Operates on	Individual bits (0s/1s)	Whole boolean expressions (true/false)
Result type	Integer	Boolean
Short-circuits?	❌ No	✅ Yes

Short-Circuit Evaluation

Definition: With `&&`, if the **first** condition is already `false`, Java doesn't bother evaluating the second condition at all — the overall result is guaranteed `false` regardless. Same logic applies to `||`: if the first condition is already `true`, the second is skipped.

```
int a = 25, b = 10, c = 15;
boolean d = (a < b) && (b < c);
```

Here `a < b` is `false` (25 is not less than 10). Since `&&` short-circuits, `b < c` is **never even evaluated** — Java already knows the final answer is `false`.

★ **Interview Tip — deliberately avoiding short-circuiting**

You can force both conditions to be evaluated even when the first is `false`, by using the **bitwise** `&` and `|` operators on boolean expressions instead of `&&` / `||`. Java allows this — the bitwise operator works fine when both operands are already `boolean`, giving the identical logical result, but *without* skipping evaluation of the second side.

This has real use later when a condition needs a **side effect** (e.g. incrementing a counter) that must always run, regardless of whether the first condition already failed.

```
boolean d = (a < b) & (b < c); // both sides ALWAYS evaluated – no short-circuit
```

 **Important:** In everyday Java code, `&&` and `||` are used **almost exclusively**. Using `&` / `|` for boolean logic is rare and mostly appears as an interview question to test whether you truly understand short-circuiting.

6. Assignment Operator

The simple `=` operator assigns the value on the right side to the variable on the left:

```
int a = 5; // "take 5, and put it into a"
```

Chained Assignment

Java allows assigning the same value to multiple variables in a single line:

```
int a = b = c = 10;
```

This evaluates **right to left**: `10` is assigned to `c` first, then `c`'s value is assigned to `b`, then `b`'s value is assigned to `a`. All three end up holding `10` — equivalent to writing three separate statements.

7. The Ternary Operator (Preview)

Java has one special operator that acts on **three operands** instead of one (unary) or two (binary) — called the **ternary operator**. Its full behavior is easiest to understand alongside `if-else` conditionals, so only the classification matters here:

Operator Type	Needs	Examples
Unary	1 operand	<code>++</code> , <code>--</code> , <code>!</code> , <code>~</code>
Binary	2 operands	<code>+</code> , <code>-</code> , <code>&&</code> , <code>==</code>
Ternary	3 operands	<code>condition ? valueIfTrue : valueIfFalse</code>

8. Operator Precedence

When an expression uses multiple operators together, Java follows a **precedence order** — similar to BODMAS/PEMDAS in math — to decide what evaluates first.

General ordering (highest to lowest):

1. Postfix (`x++` , `x--`)
2. Prefix / Unary (`++x` , `--x` , `!` , `~`)
3. Arithmetic (`*` , `/` , `%` before `+` , `-`)
4. Shift (`<<` , `>>` , `>>>`)
5. Relational (`<` , `>` , `<=` , `>=`)
6. Equality (`==` , `!=`)
7. Bitwise AND / XOR / OR (`&` , `^` , `|`)
8. Logical AND / OR (`&&` , `||`)
9. Assignment (`=` , `+=` , etc.) — **lowest precedence**, evaluated **right to left**

✓ Best Practice — the practical takeaway:

Memorizing the full precedence table isn't necessary for daily coding. In real projects, use **parentheses** `()` to make evaluation order explicit — brackets always have the **highest precedence** of all. This is clearer to read and impossible to get wrong, compared to relying on a long list of implicit rules.

```
int c = b + c * d;    // multiplication happens first (default precedence)
int c = (b + c) * d;  // parentheses force addition first
```

★ One-Page Revision

Operator Categories | Category | Operators | `|---|---|` | Arithmetic | `+` `-` `*` `/` `%` and `+=` `-=` `*=` `/=` `%=` | | Increment/Decrement | `++` `--` (prefix and postfix) | | Relational | `==` `!=` `<` `>` `<=` `>=` | | Bitwise | `&` `&|` `^` `~` | | Shift | `<<` `>>` `>>>` | | Logical | `&&` `&|` `!` | | Assignment | `=` and chained assignment | | Ternary | `? :` (3 operands) |

Key Rules

- Postfix (`j++`) uses the **old** value first, then increments; Prefix (`++j`) increments **first**, then uses the new value
- `=` assigns; `==` compares — never confuse the two
- Bitwise operators work on individual bits; logical operators work on whole boolean expressions
- `&&` / `||` **short-circuit** (skip evaluating the 2nd condition when possible); `&` / `|` do **not**
- Shift operators (`<<` , `>>` , `>>>`) only truly operate on `int` / `long` — `byte` / `short` always get promoted first
- Shift amounts wrap using `% 32` (for `int`) or `% 64` (for `long`)
- `>>` preserves sign (fills with original MSB); `>>>` always fills with `0`
- Parentheses `()` have the highest precedence — use them instead of memorizing precedence tables



Practice Questions

Basic

1. What's the difference between `a = b` and `a == b` ?
2. What does the modulus operator (`%`) return?
3. What are the three "arities" of operators in Java, and give one example of each?

Medium 4. If `int j = 9;` , what are the final values of `j` and `k` after `int k = j++;` ? 5. Write the truth table for the bitwise XOR (`^`) operator. 6. Why does `byte b = 8; b = (byte)(b << 1);` need an explicit cast?

Interview 7. Explain, with binary, why repeatedly left-shifting a `byte` eventually produces a negative number, then `0` . 8. Why do `&&` and `||` short-circuit, but `&` and `|` do not? Give a scenario where you'd deliberately want to

avoid short-circuiting. 9. What is the difference between `>>` and `>>>`, and when would you specifically want `>>>`?
10. Why is `i << 32` equivalent to `i << 0`?

Quick FAQ

Do I need to memorize the operator precedence table? No — using parentheses to make order explicit is the practical, error-proof approach used in real projects.

Is `&` ever actually used on booleans in real code? Rarely — almost all production code uses `&&` / `||`. The bitwise-on-boolean trick is mostly an interview topic.

Why does shifting a `byte` sometimes give unexpected results? Because Java always promotes it to `int` before shifting, then truncates back to 8 bits — so bits that "disappear" from the `byte` view are actually still present in the full 32-bit calculation.